🤖 CodeAlpha AI Internship

Complete Solutions for All Tasks

✅ TASK 1: Language Translation Tool

📋 Task Requirements

- Create a user interface for text input with source & target language selection
- Use a translation API (Google Translate API or Microsoft Translator)
- Process input and display translated text
- Optional: Add copy button or text-to-speech feature

🛠️ Solution Overview

We'll create a Flask-based web application with a clean UI that uses the Google Translate API (via the free `googletrans` library) for translation. The solution includes text-to-speech and copy functionality.

📁 Project Structure

```
CodeAlpha_LanguageTranslation/
├── app.py
├── requirements.txt
├── templates/
```

```
│ └── index.html
└── static/
    └── style.css
```

1. requirements.txt

```
flask==2.3.3
googletrans==4.0.0-rc1
gTTS==2.3.2
```

2. app.py (Main Application)

```
from flask import Flask, render_template, request, jsonify
from googletrans import Translator, LANGUAGES
from gtts import gTTS
import base64
import io

app = Flask(__name__)
translator = Translator()

@app.route('/')
def home():
    # Get all available languages
    languages = {code: name.capitalize() for code, name in LANGUAGES.items()}
    return render_template('index.html', languages=languages)

@app.route('/translate', methods=['POST'])
def translate():
    try:
        data = request.get_json()
        text = data.get('text', '')
        source_lang = data.get('source', 'auto')
        target_lang = data.get('target', 'en')

        if not text.strip():
            return jsonify({'error': 'Please enter text to translate'}), 400

        # Perform translation
        if source_lang == 'auto':
            result = translator.translate(text, dest=target_lang)
```

```
        else:
            result = translator.translate(text, src=source_lang, dest=target_lang)

        return jsonify({
            'translated_text': result.text,
            'detected_language': LANGUAGES.get(result.src,
result.src).capitalize(),
            'source_language': result.src
        })
    except Exception as e:
        return jsonify({'error': str(e)}), 500

@app.route('/text-to-speech', methods=['POST'])
def text_to_speech():
    try:
        data = request.get_json()
        text = data.get('text', '')
        lang = data.get('lang', 'en')

        if not text.strip():
            return jsonify({'error': 'No text provided'}), 400

        # Generate speech
        tts = gTTS(text=text, lang=lang)
        audio_buffer = io.BytesIO()
        tts.write_to_fp(audio_buffer)
        audio_buffer.seek(0)

        # Encode to base64 for frontend playback
        audio_base64 = base64.b64encode(audio_buffer.read()).decode('utf-8')

        return jsonify({'audio': audio_base64})
    except Exception as e:
        return jsonify({'error': str(e)}), 500

if __name__ == '__main__':
    app.run(debug=True, port=5000)
```

3. templates/index.html

```
<!DOCTYPE html>
<html lang="en">
```

```
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Language Translation Tool - CodeAlpha</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body>
    <div class="container">
        <h1>🌐 Language Translation Tool</h1>
        <p class="subtitle">CodeAlpha AI Internship - Task 1</p>

        <div class="translation-box">
            <div class="input-section">
                <div class="lang-select">
                    <label>Source Language:</label>
                    <select id="sourceLang">
                        <option value="auto">Auto Detect</option>
                        {% for code, name in languages.items() %}
                        <option value="{{ code }}">{{ name }}</option>
                        {% endfor %}
                    </select>
                </div>
                <textarea id="sourceText" placeholder="Enter text to translate...">
</textarea>
                <div class="btn-group">
                    <button onclick="speakText('source')" class="btn-secondary">🔊
Listen</button>
                </div>
            </div>

            <div class="swap-btn" onclick="swapLanguages()">⇄</div>

            <div class="output-section">
                <div class="lang-select">
                    <label>Target Language:</label>
                    <select id="targetLang">
                        {% for code, name in languages.items() %}
                        <option value="{{ code }}" {% if code == 'en' %}selected{%
endif %}>{{ name }}</option>
                        {% endfor %}
                    </select>
                </div>
                <textarea id="translatedText" readonly placeholder="Translation
will appear here..."></textarea>
                <div class="btn-group">
```

```
                    <button onclick="copyText()" class="btn-secondary">📋
Copy</button>
                    <button onclick="speakText('target')" class="btn-secondary">🔊
Listen</button>
                </div>
            </div>
        </div>

        <button id="translateBtn" onclick="translateText()">Translate</button>
        <p id="detectedLang" class="detected-lang"></p>
    </div>

    <script>
        let detectedSourceLang = 'en';

        async function translateText() {
            const text = document.getElementById('sourceText').value;
            const sourceLang = document.getElementById('sourceLang').value;
            const targetLang = document.getElementById('targetLang').value;

            if (!text.trim()) {
                alert('Please enter text to translate');
                return;
            }

            document.getElementById('translateBtn').textContent = 'Translating...';
            document.getElementById('translateBtn').disabled = true;

            try {
                const response = await fetch('/translate', {
                    method: 'POST',
                    headers: { 'Content-Type': 'application/json' },
                    body: JSON.stringify({ text, source: sourceLang, target:
targetLang })
                });

                const data = await response.json();

                if (data.error) {
                    alert(data.error);
                } else {
                    document.getElementById('translatedText').value =
data.translated_text;
                    document.getElementById('detectedLang').textContent =
                        `Detected Language: ${data.detected_language}`;
```

```
                    detectedSourceLang = data.source_language;
                }
            } catch (error) {
                alert('Translation failed. Please try again.');
            }

            document.getElementById('translateBtn').textContent = 'Translate';
            document.getElementById('translateBtn').disabled = false;
        }

        async function speakText(type) {
            const text = type === 'source'
                ? document.getElementById('sourceText').value
                : document.getElementById('translatedText').value;
            const lang = type === 'source'
                ? (document.getElementById('sourceLang').value === 'auto' ?
detectedSourceLang : document.getElementById('sourceLang').value)
                : document.getElementById('targetLang').value;

            if (!text.trim()) return;

            try {
                const response = await fetch('/text-to-speech', {
                    method: 'POST',
                    headers: { 'Content-Type': 'application/json' },
                    body: JSON.stringify({ text, lang })
                });

                const data = await response.json();
                if (data.audio) {
                    const audio = new Audio('data:audio/mp3;base64,' + data.audio);
                    audio.play();
                }
            } catch (error) {
                console.error('TTS failed:', error);
            }
        }

        function copyText() {
            const text = document.getElementById('translatedText').value;
            navigator.clipboard.writeText(text).then(() => {
                alert('Copied to clipboard!');
            });
        }
```

```
        function swapLanguages() {
            const sourceLang = document.getElementById('sourceLang');
            const targetLang = document.getElementById('targetLang');
            const sourceText = document.getElementById('sourceText');
            const translatedText = document.getElementById('translatedText');

            if (sourceLang.value !== 'auto') {
                const tempLang = sourceLang.value;
                sourceLang.value = targetLang.value;
                targetLang.value = tempLang;
            }

            const tempText = sourceText.value;
            sourceText.value = translatedText.value;
            translatedText.value = tempText;
        }

        // Auto-translate on Enter key
        document.getElementById('sourceText').addEventListener('keydown', (e) => {
            if (e.key === 'Enter' && e.ctrlKey) {
                translateText();
            }
        });
    </script>
</body>
</html>
```

4. static/style.css

```
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}

body {
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
    min-height: 100vh;
    display: flex;
    justify-content: center;
    align-items: center;
```

```
    padding: 20px;
}

.container {
    background: white;
    border-radius: 20px;
    padding: 40px;
    box-shadow: 0 20px 60px rgba(0,0,0,0.3);
    max-width: 900px;
    width: 100%;
}

h1 {
    text-align: center;
    color: #333;
    margin-bottom: 5px;
}

.subtitle {
    text-align: center;
    color: #666;
    margin-bottom: 30px;
}

.translation-box {
    display: flex;
    gap: 20px;
    align-items: stretch;
    margin-bottom: 20px;
}

.input-section, .output-section {
    flex: 1;
    display: flex;
    flex-direction: column;
}

.lang-select {
    margin-bottom: 10px;
}

.lang-select label {
    display: block;
    margin-bottom: 5px;
    color: #555;
```

```
    font-weight: 500;
}

.lang-select select {
    width: 100%;
    padding: 10px;
    border: 2px solid #e0e0e0;
    border-radius: 8px;
    font-size: 14px;
}

textarea {
    flex: 1;
    min-height: 150px;
    padding: 15px;
    border: 2px solid #e0e0e0;
    border-radius: 10px;
    font-size: 16px;
    resize: none;
    transition: border-color 0.3s;
}

textarea:focus {
    outline: none;
    border-color: #667eea;
}

.swap-btn {
    display: flex;
    align-items: center;
    justify-content: center;
    width: 40px;
    height: 40px;
    background: #667eea;
    color: white;
    border-radius: 50%;
    cursor: pointer;
    font-size: 20px;
    align-self: center;
    transition: transform 0.3s;
}

.swap-btn:hover {
    transform: rotate(180deg);
}
```

```
.btn-group {
    display: flex;
    gap: 10px;
    margin-top: 10px;
}

.btn-secondary {
    padding: 8px 15px;
    border: none;
    background: #f0f0f0;
    border-radius: 6px;
    cursor: pointer;
    font-size: 14px;
    transition: background 0.3s;
}

.btn-secondary:hover {
    background: #e0e0e0;
}

#translateBtn {
    width: 100%;
    padding: 15px;
    background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
    color: white;
    border: none;
    border-radius: 10px;
    font-size: 18px;
    font-weight: 600;
    cursor: pointer;
    transition: transform 0.2s, box-shadow 0.2s;
}

#translateBtn:hover {
    transform: translateY(-2px);
    box-shadow: 0 5px 20px rgba(102, 126, 234, 0.4);
}

#translateBtn:disabled {
    opacity: 0.7;
    cursor: not-allowed;
}

.detected-lang {
```

```
    text-align: center;
    margin-top: 15px;
    color: #666;
    font-style: italic;
}

@media (max-width: 768px) {
    .translation-box {
        flex-direction: column;
    }
    .swap-btn {
        transform: rotate(90deg);
    }
    .swap-btn:hover {
        transform: rotate(270deg);
    }
}
```

🚀 How to Run

1. Install dependencies: `pip install -r requirements.txt`
2. Run the application: `python app.py`
3. Open browser at: `http://localhost:5000`

> ✨ **Features Implemented:**
>
> - Auto language detection
> - 100+ languages supported
> - Text-to-speech for both source and translated text
> - Copy to clipboard functionality
> - Language swap button
> - Responsive design

✅ TASK 2: Chatbot for FAQs

📋 Task Requirements

- Collect FAQs related to a topic or product
- Preprocess text using NLP libraries (NLTK or SpaCy)
- Match user questions using cosine similarity or intent matching
- Display best matching answer as chatbot response
- Optional: Create a simple chat UI

🛠️ Solution Overview

We'll create an NLP-powered FAQ chatbot using TF-IDF vectorization and cosine similarity for question matching. The chatbot includes a web-based chat interface.

📁 Project Structure

```
CodeAlpha_FAQChatbot/
├── app.py
├── chatbot.py
├── faqs.json
├── requirements.txt
└── templates/
    └── chat.html
```

1. requirements.txt

```
flask==2.3.3
nltk==3.8.1
```

```
scikit-learn==1.3.0
numpy==1.24.3
```

2. faqs.json (Sample FAQ Database)

```
{
    "faqs": [
        {
            "question": "What are your business hours?",
            "answer": "We are open Monday to Friday, 9 AM to 6 PM. We are closed on
weekends and public holidays.",
            "keywords": ["hours", "open", "time", "business", "working"]
        },
        {
            "question": "How can I reset my password?",
            "answer": "To reset your password: 1) Click 'Forgot Password' on the
login page, 2) Enter your email, 3) Check your inbox for a reset link, 4) Create a
new password.",
            "keywords": ["password", "reset", "forgot", "change", "login"]
        },
        {
            "question": "What payment methods do you accept?",
            "answer": "We accept Visa, MasterCard, American Express, PayPal, and
bank transfers. All transactions are secured with SSL encryption.",
            "keywords": ["payment", "pay", "credit", "card", "methods", "visa",
"paypal"]
        },
        {
            "question": "How do I track my order?",
            "answer": "You can track your order by logging into your account and
clicking 'My Orders'. You'll see real-time tracking information there. You can also
use the tracking number sent to your email.",
            "keywords": ["track", "order", "shipping", "delivery", "status"]
        },
        {
            "question": "What is your return policy?",
            "answer": "We offer a 30-day return policy for unused items in original
packaging. To initiate a return, contact our support team with your order number.",
            "keywords": ["return", "refund", "policy", "exchange", "money back"]
        },
        {
            "question": "How do I contact customer support?",
            "answer": "You can reach us via: Email: support@example.com, Phone: 1-
```

```
800-123-4567 (Mon-Fri 9AM-6PM), or Live Chat on our website.",
            "keywords": ["contact", "support", "help", "customer service", "phone",
"email"]
        },
        {
            "question": "Do you offer international shipping?",
            "answer": "Yes! We ship to over 100 countries. International shipping
typically takes 7-14 business days. Shipping costs vary by location.",
            "keywords": ["international", "shipping", "worldwide", "abroad",
"countries"]
        },
        {
            "question": "How do I create an account?",
            "answer": "Click 'Sign Up' on our homepage, enter your email and create
a password. You'll receive a verification email to activate your account.",
            "keywords": ["create", "account", "sign up", "register", "new user"]
        },
        {
            "question": "What is your privacy policy?",
            "answer": "We take privacy seriously. We never sell your data to third
parties. Your information is encrypted and stored securely. Read our full privacy
policy on our website.",
            "keywords": ["privacy", "data", "information", "secure", "policy"]
        },
        {
            "question": "Do you have a mobile app?",
            "answer": "Yes! Our mobile app is available on both iOS (App Store) and
Android (Google Play). Search for our company name to download it free.",
            "keywords": ["app", "mobile", "ios", "android", "download", "phone"]
        }
    ]
}
```

3. chatbot.py (NLP Chatbot Engine)

```
import json
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
```

```
import numpy as np
import re

# Download required NLTK data
nltk.download('punkt', quiet=True)
nltk.download('stopwords', quiet=True)
nltk.download('wordnet', quiet=True)
nltk.download('punkt_tab', quiet=True)

class FAQChatbot:
    def __init__(self, faq_file='faqs.json'):
        self.lemmatizer = WordNetLemmatizer()
        self.stop_words = set(stopwords.words('english'))
        self.load_faqs(faq_file)
        self.build_vectorizer()

    def load_faqs(self, faq_file):
        """Load FAQ data from JSON file"""
        with open(faq_file, 'r') as f:
            data = json.load(f)
        self.faqs = data['faqs']
        self.questions = [faq['question'] for faq in self.faqs]
        self.answers = [faq['answer'] for faq in self.faqs]
        self.keywords = [faq.get('keywords', []) for faq in self.faqs]

    def preprocess_text(self, text):
        """Clean and preprocess text using NLP techniques"""
        # Convert to lowercase
        text = text.lower()
        # Remove special characters and digits
        text = re.sub(r'[^a-zA-Z\s]', '', text)
        # Tokenize
        tokens = word_tokenize(text)
        # Remove stopwords and lemmatize
        processed_tokens = [
            self.lemmatizer.lemmatize(token)
            for token in tokens
            if token not in self.stop_words and len(token) > 2
        ]
        return ' '.join(processed_tokens)

    def build_vectorizer(self):
        """Build TF-IDF vectorizer from FAQ questions"""
        # Preprocess all questions
        processed_questions = [self.preprocess_text(q) for q in self.questions]
```

```
        # Also include keywords in the vectorization
        combined_texts = []
        for i, q in enumerate(processed_questions):
            keywords_text = ' '.join(self.keywords[i]) if self.keywords[i] else ''
            combined_texts.append(f"{q} {keywords_text}")

        # Build TF-IDF vectorizer
        self.vectorizer = TfidfVectorizer()
        self.faq_vectors = self.vectorizer.fit_transform(combined_texts)

    def find_best_match(self, user_question):
        """Find the most similar FAQ to the user's question"""
        # Preprocess user question
        processed_question = self.preprocess_text(user_question)

        if not processed_question.strip():
            return None, 0.0

        # Vectorize user question
        user_vector = self.vectorizer.transform([processed_question])

        # Calculate cosine similarity with all FAQs
        similarities = cosine_similarity(user_vector, self.faq_vectors)[0]

        # Find the best match
        best_idx = np.argmax(similarities)
        best_score = similarities[best_idx]

        return best_idx, best_score

    def get_response(self, user_question):
        """Generate chatbot response for user question"""
        # Handle greetings
        greetings = ['hi', 'hello', 'hey', 'good morning', 'good afternoon', 'good
evening']
        if user_question.lower().strip() in greetings:
            return {
                'answer': "Hello! 👋 I'm your FAQ assistant. How can I help you
today?",
                'confidence': 1.0,
                'matched_question': None
            }

        # Handle thanks
```

```
        thanks = ['thanks', 'thank you', 'thx', 'ty']
        if any(t in user_question.lower() for t in thanks):
            return {
                'answer': "You're welcome! 😊 Is there anything else I can help you
with?",
                'confidence': 1.0,
                'matched_question': None
            }

        # Find best matching FAQ
        best_idx, confidence = self.find_best_match(user_question)

        # Set confidence threshold
        CONFIDENCE_THRESHOLD = 0.2

        if confidence >= CONFIDENCE_THRESHOLD:
            return {
                'answer': self.answers[best_idx],
                'confidence': round(confidence, 2),
                'matched_question': self.questions[best_idx]
            }
        else:
            return {
                'answer': "I'm sorry, I couldn't find a relevant answer to your
question. Please try rephrasing or contact our support team for assistance.",
                'confidence': round(confidence, 2),
                'matched_question': None
            }

    def get_all_faqs(self):
        """Return all FAQs for display"""
        return [{'question': q, 'answer': a} for q, a in zip(self.questions,
self.answers)]


# Test the chatbot
if __name__ == '__main__':
    chatbot = FAQChatbot()

    test_questions = [
        "What time do you open?",
        "I forgot my password",
        "Can I pay with PayPal?",
        "Where is my package?",
        "How do I return an item?"
```

```
    ]

    print("=" * 50)
    print("FAQ Chatbot Test")
    print("=" * 50)

    for question in test_questions:
        response = chatbot.get_response(question)
        print(f"\nQ: {question}")
        print(f"A: {response['answer']}")
        print(f"Confidence: {response['confidence']}")
```

4. app.py (Flask Web Application)

```
from flask import Flask, render_template, request, jsonify
from chatbot import FAQChatbot

app = Flask(__name__)
chatbot = FAQChatbot()

@app.route('/')
def home():
    faqs = chatbot.get_all_faqs()
    return render_template('chat.html', faqs=faqs)

@app.route('/chat', methods=['POST'])
def chat():
    data = request.get_json()
    user_message = data.get('message', '')

    if not user_message.strip():
        return jsonify({'error': 'Empty message'}), 400

    response = chatbot.get_response(user_message)
    return jsonify(response)

@app.route('/faqs', methods=['GET'])
def get_faqs():
    return jsonify(chatbot.get_all_faqs())
```

```
if __name__ == '__main__':
    app.run(debug=True, port=5000)
```

5. templates/chat.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>FAQ Chatbot - CodeAlpha</title>
    <style>
        * { margin: 0; padding: 0; box-sizing: border-box; }
        body {
            font-family: 'Segoe UI', Tahoma, sans-serif;
            background: linear-gradient(135deg, #1a1a2e 0%, #16213e 100%);
            min-height: 100vh;
            display: flex;
            justify-content: center;
            align-items: center;
            padding: 20px;
        }
        .chat-container {
            width: 100%;
            max-width: 500px;
            background: white;
            border-radius: 20px;
            box-shadow: 0 20px 50px rgba(0,0,0,0.3);
            overflow: hidden;
        }
        .chat-header {
            background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
            color: white;
            padding: 20px;
            text-align: center;
        }
        .chat-header h1 { font-size: 1.5em; margin-bottom: 5px; }
        .chat-header p { opacity: 0.9; font-size: 0.9em; }
        .chat-messages {
            height: 400px;
            overflow-y: auto;
            padding: 20px;
            background: #f8f9fa;
```

```
        }
        .message {
            margin-bottom: 15px;
            display: flex;
            flex-direction: column;
        }
        .message.user { align-items: flex-end; }
        .message.bot { align-items: flex-start; }
        .message-content {
            max-width: 80%;
            padding: 12px 18px;
            border-radius: 18px;
            font-size: 14px;
            line-height: 1.5;
        }
        .user .message-content {
            background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
            color: white;
            border-bottom-right-radius: 5px;
        }
        .bot .message-content {
            background: white;
            color: #333;
            border-bottom-left-radius: 5px;
            box-shadow: 0 2px 5px rgba(0,0,0,0.1);
        }
        .confidence {
            font-size: 11px;
            color: #888;
            margin-top: 5px;
        }
        .chat-input {
            display: flex;
            padding: 15px;
            background: white;
            border-top: 1px solid #eee;
        }
        .chat-input input {
            flex: 1;
            padding: 12px 18px;
            border: 2px solid #e0e0e0;
            border-radius: 25px;
            font-size: 14px;
            outline: none;
            transition: border-color 0.3s;
```

```
        }
        .chat-input input:focus { border-color: #667eea; }
        .chat-input button {
            margin-left: 10px;
            padding: 12px 25px;
            background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
            color: white;
            border: none;
            border-radius: 25px;
            cursor: pointer;
            font-weight: 600;
            transition: transform 0.2s;
        }
        .chat-input button:hover { transform: scale(1.05); }
        .quick-questions {
            padding: 10px 15px;
            background: #f0f0f0;
            display: flex;
            flex-wrap: wrap;
            gap: 8px;
        }
        .quick-btn {
            padding: 6px 12px;
            background: white;
            border: 1px solid #ddd;
            border-radius: 15px;
            font-size: 12px;
            cursor: pointer;
            transition: all 0.2s;
        }
        .quick-btn:hover {
            background: #667eea;
            color: white;
            border-color: #667eea;
        }
        .typing-indicator {
            display: none;
            padding: 10px 15px;
        }
        .typing-indicator span {
            display: inline-block;
            width: 8px;
            height: 8px;
            background: #667eea;
            border-radius: 50%;
```

```
            margin-right: 5px;
            animation: bounce 1.4s infinite ease-in-out;
        }
        .typing-indicator span:nth-child(1) { animation-delay: 0s; }
        .typing-indicator span:nth-child(2) { animation-delay: 0.2s; }
        .typing-indicator span:nth-child(3) { animation-delay: 0.4s; }
        @keyframes bounce {
            0%, 80%, 100% { transform: scale(0); }
            40% { transform: scale(1); }
        }
    </style>
</head>
<body>
    <div class="chat-container">
        <div class="chat-header">
            <h1>🤖 FAQ Chatbot</h1>
            <p>CodeAlpha AI Internship - Task 2</p>
        </div>

        <div class="quick-questions">
            <button class="quick-btn" onclick="askQuestion('What are your business
hours?')">Business Hours</button>
            <button class="quick-btn" onclick="askQuestion('How do I reset my
password?')">Reset Password</button>
            <button class="quick-btn" onclick="askQuestion('What payment methods do
you accept?')">Payment Methods</button>
            <button class="quick-btn" onclick="askQuestion('How do I track my
order?')">Track Order</button>
        </div>

        <div class="chat-messages" id="chatMessages">
            <div class="message bot">
                <div class="message-content">
                    Hello! 👋 I'm your FAQ assistant. Ask me anything about our
services, and I'll try to help!
                </div>
            </div>
        </div>

        <div class="typing-indicator" id="typingIndicator">
            <span></span><span></span><span></span>
        </div>

        <div class="chat-input">
            <input type="text" id="userInput" placeholder="Type your question..."
```

```
onkeypress="handleKeyPress(event)">
            <button onclick="sendMessage()">Send</button>
        </div>
    </div>

    <script>
        function addMessage(content, isUser, confidence = null) {
            const messagesDiv = document.getElementById('chatMessages');
            const messageDiv = document.createElement('div');
            messageDiv.className = `message ${isUser ? 'user' : 'bot'}`;

            let html = `<div class="message-content">${content}</div>`;
            if (confidence !== null && !isUser) {
                html += `<div class="confidence">Confidence: ${(confidence *
100).toFixed(0)}%</div>`;
            }

            messageDiv.innerHTML = html;
            messagesDiv.appendChild(messageDiv);
            messagesDiv.scrollTop = messagesDiv.scrollHeight;
        }

        async function sendMessage() {
            const input = document.getElementById('userInput');
            const message = input.value.trim();

            if (!message) return;

            addMessage(message, true);
            input.value = '';

            document.getElementById('typingIndicator').style.display = 'block';

            try {
                const response = await fetch('/chat', {
                    method: 'POST',
                    headers: { 'Content-Type': 'application/json' },
                    body: JSON.stringify({ message })
                });

                const data = await response.json();
                document.getElementById('typingIndicator').style.display = 'none';
                addMessage(data.answer, false, data.confidence);

            } catch (error) {
```

```
                document.getElementById('typingIndicator').style.display = 'none';
                addMessage('Sorry, something went wrong. Please try again.',
false);
            }
        }

        function askQuestion(question) {
            document.getElementById('userInput').value = question;
            sendMessage();
        }

        function handleKeyPress(event) {
            if (event.key === 'Enter') sendMessage();
        }
    </script>
</body>
</html>
```

🚀 How to Run

1. Install dependencies: `pip install -r requirements.txt`
2. Run the application: `python app.py`
3. Open browser at: `http://localhost:5000`

> ✨ **Features Implemented:**
>
> - NLP preprocessing with NLTK (tokenization, lemmatization, stopword removal)
> - TF-IDF vectorization for text representation
> - Cosine similarity for question matching
> - Confidence score display
> - Quick question buttons
> - Typing indicator animation
> - Greeting and thanks handling

✅ TASK 3: Music Generation with AI

📋 Task Requirements

- Collect MIDI music data for training
- Preprocess data into note sequences (using music21)
- Build deep learning model (RNN/LSTM or GANs)
- Train model to generate new music sequences
- Convert generated sequences to MIDI and save as audio

🛠️ Solution Overview

We'll create an LSTM-based music generation system that learns patterns from MIDI files and generates new musical compositions.

📁 Project Structure

```
CodeAlpha_MusicGeneration/
├── music_generator.py
├── train.py
├── generate.py
├── requirements.txt
├── midi_files/ (place training MIDI files here)
└── output/ (generated music saved here)
```

1. requirements.txt

```
music21==9.1.0
numpy==1.24.3
tensorflow==2.13.0
keras==2.13.0
pretty_midi==0.2.10
```

2. music_generator.py (Main Music Generation Module)

```
import numpy as np
from music21 import converter, instrument, note, chord, stream
from tensorflow.keras.models import Sequential, load_model
from tensorflow.keras.layers import LSTM, Dense, Dropout, Activation,
BatchNormalization
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping
from tensorflow.keras.utils import to_categorical
import os
import pickle
import glob

class MusicGenerator:
    def __init__(self, sequence_length=100):
        self.sequence_length = sequence_length
        self.notes = []
        self.note_to_int = {}
        self.int_to_note = {}
        self.n_vocab = 0
        self.model = None

    def get_notes_from_midi(self, midi_folder='midi_files'):
        """Extract notes and chords from MIDI files"""
        notes = []

        # Get all MIDI files
        midi_files = glob.glob(os.path.join(midi_folder, '*.mid')) + \
                     glob.glob(os.path.join(midi_folder, '*.midi'))

        if not midi_files:
            print(f"No MIDI files found in {midi_folder}")
            print("Creating sample MIDI for demonstration...")
            self._create_sample_midi(midi_folder)
            midi_files = glob.glob(os.path.join(midi_folder, '*.mid'))
```

```
        print(f"Processing {len(midi_files)} MIDI files...")

        for file in midi_files:
            try:
                midi = converter.parse(file)
                print(f"Parsing: {file}")

                notes_to_parse = None

                # Handle different MIDI structures
                try:
                    # File has instrument parts
                    parts = instrument.partitionByInstrument(midi)
                    notes_to_parse = parts.parts[0].recurse()
                except:
                    # File has notes in flat structure
                    notes_to_parse = midi.flat.notes

                for element in notes_to_parse:
                    if isinstance(element, note.Note):
                        notes.append(str(element.pitch))
                    elif isinstance(element, chord.Chord):
                        notes.append('.'.join(str(n) for n in element.normalOrder))

            except Exception as e:
                print(f"Error processing {file}: {e}")
                continue

        self.notes = notes
        print(f"Total notes extracted: {len(notes)}")
        return notes

    def _create_sample_midi(self, folder):
        """Create a sample MIDI file for demonstration"""
        os.makedirs(folder, exist_ok=True)

        # Create a simple melody
        s = stream.Stream()

        # C major scale pattern repeated with variations
        patterns = [
            ['C4', 'E4', 'G4', 'C5', 'G4', 'E4', 'C4', 'D4'],
            ['E4', 'G4', 'B4', 'E5', 'B4', 'G4', 'E4', 'F4'],
            ['G4', 'B4', 'D5', 'G5', 'D5', 'B4', 'G4', 'A4'],
            ['C4', 'D4', 'E4', 'F4', 'G4', 'A4', 'B4', 'C5'],
```

```
        ]

        for _ in range(5):  # Repeat patterns
            for pattern in patterns:
                for p in pattern:
                    n = note.Note(p)
                    n.duration.quarterLength = 0.5
                    s.append(n)

        # Save the MIDI file
        s.write('midi', fp=os.path.join(folder, 'sample.mid'))
        print(f"Created sample MIDI file in {folder}")

    def prepare_sequences(self):
        """Prepare input sequences for training"""
        # Get unique notes
        unique_notes = sorted(set(self.notes))
        self.n_vocab = len(unique_notes)

        print(f"Unique notes/chords: {self.n_vocab}")

        # Create mappings
        self.note_to_int = {note: num for num, note in enumerate(unique_notes)}
        self.int_to_note = {num: note for num, note in enumerate(unique_notes)}

        # Create input sequences and outputs
        network_input = []
        network_output = []

        for i in range(len(self.notes) - self.sequence_length):
            seq_in = self.notes[i:i + self.sequence_length]
            seq_out = self.notes[i + self.sequence_length]

            network_input.append([self.note_to_int[char] for char in seq_in])
            network_output.append(self.note_to_int[seq_out])

        n_patterns = len(network_input)
        print(f"Total training sequences: {n_patterns}")

        # Reshape for LSTM [samples, time steps, features]
        network_input = np.reshape(network_input, (n_patterns,
self.sequence_length, 1))
        # Normalize
        network_input = network_input / float(self.n_vocab)
        # One-hot encode output
```

```
        network_output = to_categorical(network_output, num_classes=self.n_vocab)

        return network_input, network_output

    def build_model(self, network_input):
        """Build the LSTM model"""
        model = Sequential([
            LSTM(256, input_shape=(network_input.shape[1], network_input.shape[2]),
                 return_sequences=True),
            Dropout(0.3),
            BatchNormalization(),

            LSTM(256, return_sequences=True),
            Dropout(0.3),
            BatchNormalization(),

            LSTM(256),
            Dropout(0.3),
            BatchNormalization(),

            Dense(256, activation='relu'),
            Dropout(0.3),

            Dense(self.n_vocab, activation='softmax')
        ])

        model.compile(loss='categorical_crossentropy', optimizer='adam')
        self.model = model

        print(model.summary())
        return model

    def train(self, network_input, network_output, epochs=100, batch_size=64):
        """Train the model"""
        # Callbacks
        checkpoint = ModelCheckpoint(
            'music_model_weights.keras',
            monitor='loss',
            verbose=1,
            save_best_only=True,
            mode='min'
        )

        early_stop = EarlyStopping(
            monitor='loss',
```

```
            patience=10,
            verbose=1,
            restore_best_weights=True
        )

        history = self.model.fit(
            network_input,
            network_output,
            epochs=epochs,
            batch_size=batch_size,
            callbacks=[checkpoint, early_stop],
            verbose=1
        )

        return history

    def generate_music(self, seed_sequence=None, num_notes=500, temperature=1.0):
        """Generate new music using the trained model"""
        if self.model is None:
            raise ValueError("Model not trained or loaded")

        # Use random seed if not provided
        if seed_sequence is None:
            start = np.random.randint(0, len(self.notes) - self.sequence_length)
            pattern = [self.note_to_int[n] for n in self.notes[start:start +
self.sequence_length]]
        else:
            pattern = seed_sequence

        generated_notes = []

        print(f"Generating {num_notes} notes...")

        for i in range(num_notes):
            # Prepare input
            prediction_input = np.reshape(pattern, (1, len(pattern), 1))
            prediction_input = prediction_input / float(self.n_vocab)

            # Get prediction
            prediction = self.model.predict(prediction_input, verbose=0)

            # Apply temperature for creativity
            prediction = np.log(prediction + 1e-7) / temperature
            prediction = np.exp(prediction) / np.sum(np.exp(prediction))
```

```
            # Sample from the distribution
            index = np.random.choice(len(prediction[0]), p=prediction[0])

            result = self.int_to_note[index]
            generated_notes.append(result)

            # Update pattern for next prediction
            pattern.append(index)
            pattern = pattern[1:]

            if (i + 1) % 100 == 0:
                print(f"Generated {i + 1}/{num_notes} notes")

        return generated_notes

    def notes_to_midi(self, notes, output_file='output/generated_music.mid'):
        """Convert generated notes to MIDI file"""
        os.makedirs(os.path.dirname(output_file), exist_ok=True)

        output_notes = []
        offset = 0

        for pattern in notes:
            # Pattern is a chord
            if ('.' in pattern) or pattern.isdigit():
                notes_in_chord = pattern.split('.')
                chord_notes = []
                for current_note in notes_in_chord:
                    try:
                        new_note = note.Note(int(current_note))
                        new_note.storedInstrument = instrument.Piano()
                        chord_notes.append(new_note)
                    except:
                        continue
                if chord_notes:
                    new_chord = chord.Chord(chord_notes)
                    new_chord.offset = offset
                    output_notes.append(new_chord)
            # Pattern is a single note
            else:
                try:
                    new_note = note.Note(pattern)
                    new_note.offset = offset
                    new_note.storedInstrument = instrument.Piano()
                    output_notes.append(new_note)
```

```
                except:
                    continue

            # Increase offset (time between notes)
            offset += 0.5

        midi_stream = stream.Stream(output_notes)
        midi_stream.write('midi', fp=output_file)

        print(f"MIDI file saved to: {output_file}")
        return output_file

    def save_state(self, filepath='music_generator_state.pkl'):
        """Save the generator state"""
        state = {
            'notes': self.notes,
            'note_to_int': self.note_to_int,
            'int_to_note': self.int_to_note,
            'n_vocab': self.n_vocab,
            'sequence_length': self.sequence_length
        }
        with open(filepath, 'wb') as f:
            pickle.dump(state, f)
        print(f"State saved to {filepath}")

    def load_state(self, filepath='music_generator_state.pkl'):
        """Load the generator state"""
        with open(filepath, 'rb') as f:
            state = pickle.load(f)
        self.notes = state['notes']
        self.note_to_int = state['note_to_int']
        self.int_to_note = state['int_to_note']
        self.n_vocab = state['n_vocab']
        self.sequence_length = state['sequence_length']
        print(f"State loaded from {filepath}")

    def load_trained_model(self, model_path='music_model_weights.keras'):
        """Load a pre-trained model"""
        self.model = load_model(model_path)
        print(f"Model loaded from {model_path}")
```

3. train.py (Training Script)

```
from music_generator import MusicGenerator

def main():
    print("=" * 50)
    print("Music Generation AI - Training")
    print("=" * 50)

    # Initialize generator
    generator = MusicGenerator(sequence_length=100)

    # Step 1: Extract notes from MIDI files
    print("\n[Step 1] Extracting notes from MIDI files...")
    notes = generator.get_notes_from_midi('midi_files')

    if len(notes) < generator.sequence_length + 1:
        print("Not enough notes for training. Add more MIDI files.")
        return

    # Step 2: Prepare sequences
    print("\n[Step 2] Preparing training sequences...")
    network_input, network_output = generator.prepare_sequences()

    # Step 3: Build model
    print("\n[Step 3] Building LSTM model...")
    generator.build_model(network_input)

    # Step 4: Train
    print("\n[Step 4] Training model (this may take a while)...")
    generator.train(
        network_input,
        network_output,
        epochs=50,  # Reduce for faster training
        batch_size=64
    )

    # Step 5: Save state
    print("\n[Step 5] Saving generator state...")
    generator.save_state()

    print("\n" + "=" * 50)
    print("Training complete! Run generate.py to create music.")
    print("=" * 50)
```

```
if __name__ == '__main__':
    main()
```

4. generate.py (Generation Script)

```
from music_generator import MusicGenerator
import os

def main():
    print("=" * 50)
    print("Music Generation AI - Generate New Music")
    print("=" * 50)

    generator = MusicGenerator(sequence_length=100)

    # Load saved state
    try:
        generator.load_state('music_generator_state.pkl')
        generator.load_trained_model('music_model_weights.keras')
    except FileNotFoundError:
        print("No trained model found. Please run train.py first.")
        return

    # Generate music with different temperatures
    temperatures = [0.5, 1.0, 1.5]

    for temp in temperatures:
        print(f"\n[Generating with temperature {temp}]")

        # Generate notes
        generated_notes = generator.generate_music(
            num_notes=200,
            temperature=temp
        )

        # Convert to MIDI
        output_file = f'output/generated_music_temp_{temp}.mid'
        generator.notes_to_midi(generated_notes, output_file)

    print("\n" + "=" * 50)
    print("Music generation complete!")
    print("Check the 'output' folder for generated MIDI files.")
    print("=" * 50)
```

```
if __name__ == '__main__':
    main()
```

🚀 How to Run

1. Install dependencies: `pip install -r requirements.txt`
2. Place MIDI files in `midi_files/` folder (or use auto-generated sample)
3. Train the model: `python train.py`
4. Generate music: `python generate.py`
5. Play generated MIDI files from `output/` folder

> 📝 **Note:** Training requires a good amount of MIDI data. You can download free MIDI files from:
>
> - Classical Music MIDI Archive
> - FreeMidi.org
> - Mutopia Project

> ✨ **Features Implemented:**
>
> - MIDI file parsing with music21
> - Note sequence extraction (notes and chords)
> - 3-layer LSTM network with dropout and batch normalization
> - Temperature-controlled generation for creativity
> - Automatic sample MIDI creation for testing
> - State saving and loading for model persistence

✅ TASK 4: Object Detection and Tracking

📋 Task Requirements

- Set up real-time video input (webcam or video file)
- Use pre-trained model (YOLO or Faster R-CNN) for detection
- Process each frame to detect objects with bounding boxes
- Apply object tracking (SORT or Deep SORT)
- Display output with labels and tracking IDs in real time

🛠️ Solution Overview

We'll use YOLOv8 for object detection combined with the ByteTrack algorithm for multi-object tracking. This provides real-time performance with accurate tracking.

📁 Project Structure

```
CodeAlpha_ObjectDetection/
├── object_tracker.py
├── simple_tracker.py
├── main.py
├── requirements.txt
└── test_videos/ (optional test videos)
```

1. requirements.txt

```
opencv-python==4.8.0.76
ultralytics==8.0.196
```

```
numpy==1.24.3
scipy==1.11.2
```

2. simple_tracker.py (Custom SORT-like Tracker)

```
import numpy as np
from scipy.optimize import linear_sum_assignment

class SimpleTracker:
    """
    Simple Object Tracker using IoU matching (SORT-like implementation)
    """
    def __init__(self, max_disappeared=30, min_hits=3):
        self.next_id = 0
        self.tracked_objects = {}  # {id: {'bbox': [...], 'hits': int, 'age': int}}
        self.max_disappeared = max_disappeared
        self.min_hits = min_hits

    def compute_iou(self, box1, box2):
        """Calculate Intersection over Union between two boxes"""
        x1 = max(box1[0], box2[0])
        y1 = max(box1[1], box2[1])
        x2 = min(box1[2], box2[2])
        y2 = min(box1[3], box2[3])

        intersection = max(0, x2 - x1) * max(0, y2 - y1)

        area1 = (box1[2] - box1[0]) * (box1[3] - box1[1])
        area2 = (box2[2] - box2[0]) * (box2[3] - box2[1])

        union = area1 + area2 - intersection

        return intersection / union if union > 0 else 0

    def update(self, detections):
        """
        Update tracker with new detections
        detections: list of [x1, y1, x2, y2, confidence, class_id]
        Returns: list of [x1, y1, x2, y2, track_id, class_id]
        """
        # If no detections, increase age of all tracked objects
        if len(detections) == 0:
            for track_id in list(self.tracked_objects.keys()):
```

```
                self.tracked_objects[track_id]['age'] += 1
                if self.tracked_objects[track_id]['age'] > self.max_disappeared:
                    del self.tracked_objects[track_id]
            return []

        # If no tracked objects, register all detections
        if len(self.tracked_objects) == 0:
            for det in detections:
                self._register(det)
            return self._get_active_tracks()

        # Calculate IoU matrix between tracked objects and detections
        track_ids = list(self.tracked_objects.keys())
        iou_matrix = np.zeros((len(track_ids), len(detections)))

        for i, track_id in enumerate(track_ids):
            for j, det in enumerate(detections):
                iou_matrix[i, j] = self.compute_iou(
                    self.tracked_objects[track_id]['bbox'],
                    det[:4]
                )

        # Use Hungarian algorithm for optimal assignment
        row_indices, col_indices = linear_sum_assignment(-iou_matrix)

        matched_tracks = set()
        matched_detections = set()

        for row, col in zip(row_indices, col_indices):
            if iou_matrix[row, col] >= 0.3:  # IoU threshold
                track_id = track_ids[row]
                det = detections[col]
                self.tracked_objects[track_id]['bbox'] = det[:4]
                self.tracked_objects[track_id]['class_id'] = int(det[5])
                self.tracked_objects[track_id]['hits'] += 1
                self.tracked_objects[track_id]['age'] = 0
                matched_tracks.add(row)
                matched_detections.add(col)

        # Handle unmatched tracks (disappeared objects)
        for i, track_id in enumerate(track_ids):
            if i not in matched_tracks:
                self.tracked_objects[track_id]['age'] += 1
                if self.tracked_objects[track_id]['age'] > self.max_disappeared:
                    del self.tracked_objects[track_id]
```

```
        # Register unmatched detections as new tracks
        for j, det in enumerate(detections):
            if j not in matched_detections:
                self._register(det)

        return self._get_active_tracks()

    def _register(self, detection):
        """Register a new track"""
        self.tracked_objects[self.next_id] = {
            'bbox': detection[:4],
            'class_id': int(detection[5]),
            'hits': 1,
            'age': 0
        }
        self.next_id += 1

    def _get_active_tracks(self):
        """Return tracks that have been seen enough times"""
        tracks = []
        for track_id, obj in self.tracked_objects.items():
            if obj['hits'] >= self.min_hits:
                track = list(obj['bbox']) + [track_id, obj['class_id']]
                tracks.append(track)
        return tracks
```

3. object_tracker.py (Main Object Detection & Tracking)

```
import cv2
import numpy as np
from ultralytics import YOLO
from simple_tracker import SimpleTracker
import time

class ObjectDetectorTracker:
    def __init__(self, model_name='yolov8n.pt', confidence=0.5):
        """
        Initialize the object detector and tracker

        Args:
            model_name: YOLO model to use (yolov8n, yolov8s, yolov8m, yolov8l,
```

```
yolov8x)
            confidence: Detection confidence threshold
        """
        print(f"Loading YOLO model: {model_name}")
        self.model = YOLO(model_name)
        self.confidence = confidence
        self.tracker = SimpleTracker(max_disappeared=30, min_hits=3)

        # Colors for different track IDs (generate unique colors)
        np.random.seed(42)
        self.colors = np.random.randint(0, 255, size=(1000, 3), dtype=np.uint8)

        # COCO class names
        self.class_names = self.model.names

    def detect_and_track(self, frame):
        """
        Perform detection and tracking on a frame

        Returns:
            frame: Annotated frame
            tracks: List of tracked objects [x1, y1, x2, y2, track_id, class_id]
        """
        # Run YOLO detection
        results = self.model(frame, conf=self.confidence, verbose=False)

        # Extract detections
        detections = []
        for result in results:
            boxes = result.boxes
            if boxes is not None:
                for box in boxes:
                    x1, y1, x2, y2 = box.xyxy[0].cpu().numpy()
                    conf = box.conf[0].cpu().numpy()
                    cls = box.cls[0].cpu().numpy()
                    detections.append([x1, y1, x2, y2, conf, cls])

        # Update tracker
        tracks = self.tracker.update(detections)

        return frame, tracks

    def draw_tracks(self, frame, tracks):
        """Draw bounding boxes and labels on the frame"""
        for track in tracks:
```

```
            x1, y1, x2, y2, track_id, class_id = track
            x1, y1, x2, y2 = int(x1), int(y1), int(x2), int(y2)
            track_id = int(track_id)
            class_id = int(class_id)

            # Get color for this track ID
            color = tuple(map(int, self.colors[track_id % 1000]))

            # Draw bounding box
            cv2.rectangle(frame, (x1, y1), (x2, y2), color, 2)

            # Prepare label
            class_name = self.class_names.get(class_id, f"Class{class_id}")
            label = f"ID:{track_id} {class_name}"

            # Draw label background
            (label_width, label_height), baseline = cv2.getTextSize(
                label, cv2.FONT_HERSHEY_SIMPLEX, 0.6, 2
            )
            cv2.rectangle(
                frame,
                (x1, y1 - label_height - 10),
                (x1 + label_width, y1),
                color,
                -1
            )

            # Draw label text
            cv2.putText(
                frame, label, (x1, y1 - 5),
                cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 255, 255), 2
            )

        return frame

    def process_video(self, source=0, output_path=None, show=True):
        """
        Process video from webcam or file

        Args:
            source: 0 for webcam, or path to video file
            output_path: Path to save output video (optional)
            show: Whether to display the output
        """
        # Open video source
```

```
        cap = cv2.VideoCapture(source)

        if not cap.isOpened():
            print(f"Error: Could not open video source: {source}")
            return

        # Get video properties
        fps = int(cap.get(cv2.CAP_PROP_FPS)) or 30
        width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
        height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

        print(f"Video: {width}x{height} @ {fps} FPS")

        # Setup video writer if saving output
        writer = None
        if output_path:
            fourcc = cv2.VideoWriter_fourcc(*'mp4v')
            writer = cv2.VideoWriter(output_path, fourcc, fps, (width, height))

        frame_count = 0
        start_time = time.time()

        print("\nPress 'q' to quit, 's' to save screenshot")

        while True:
            ret, frame = cap.read()
            if not ret:
                break

            frame_count += 1

            # Detect and track objects
            frame, tracks = self.detect_and_track(frame)

            # Draw tracked objects
            frame = self.draw_tracks(frame, tracks)

            # Calculate and display FPS
            elapsed_time = time.time() - start_time
            fps_display = frame_count / elapsed_time if elapsed_time > 0 else 0

            # Draw info overlay
            info_text = f"FPS: {fps_display:.1f} | Objects: {len(tracks)}"
            cv2.putText(
                frame, info_text, (10, 30),
```

```
                cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0), 2
            )

            # Show frame
            if show:
                cv2.imshow("Object Detection & Tracking - CodeAlpha", frame)

            # Write frame to output video
            if writer:
                writer.write(frame)

            # Handle key presses
            key = cv2.waitKey(1) & 0xFF
            if key == ord('q'):
                break
            elif key == ord('s'):
                screenshot_path = f"screenshot_{frame_count}.jpg"
                cv2.imwrite(screenshot_path, frame)
                print(f"Screenshot saved: {screenshot_path}")

        # Cleanup
        cap.release()
        if writer:
            writer.release()
        cv2.destroyAllWindows()

        print(f"\nProcessed {frame_count} frames in {elapsed_time:.2f} seconds")
        print(f"Average FPS: {fps_display:.2f}")

    def process_single_image(self, image_path, output_path=None):
        """Process a single image"""
        frame = cv2.imread(image_path)
        if frame is None:
            print(f"Error: Could not read image: {image_path}")
            return

        # Detect (tracking doesn't make sense for single image)
        results = self.model(frame, conf=self.confidence, verbose=False)

        # Draw detections
        for result in results:
            boxes = result.boxes
            if boxes is not None:
                for i, box in enumerate(boxes):
                    x1, y1, x2, y2 = map(int, box.xyxy[0].cpu().numpy())
```

```
                    conf = float(box.conf[0].cpu().numpy())
                    cls = int(box.cls[0].cpu().numpy())

                    color = tuple(map(int, self.colors[i]))
                    class_name = self.class_names.get(cls, f"Class{cls}")

                    cv2.rectangle(frame, (x1, y1), (x2, y2), color, 2)
                    label = f"{class_name}: {conf:.2f}"
                    cv2.putText(frame, label, (x1, y1-10),
                                cv2.FONT_HERSHEY_SIMPLEX, 0.6, color, 2)

        if output_path:
            cv2.imwrite(output_path, frame)
            print(f"Output saved to: {output_path}")

        cv2.imshow("Detection Result", frame)
        cv2.waitKey(0)
        cv2.destroyAllWindows()
```

4. main.py (Main Entry Point)

```
from object_tracker import ObjectDetectorTracker
import argparse
import os

def main():
    parser = argparse.ArgumentParser(description='Object Detection and Tracking -
CodeAlpha')
    parser.add_argument('--source', type=str, default='0',
                        help='Video source: 0 for webcam, or path to video file')
    parser.add_argument('--model', type=str, default='yolov8n.pt',
                        help='YOLO model to use (yolov8n.pt, yolov8s.pt,
yolov8m.pt)')
    parser.add_argument('--confidence', type=float, default=0.5,
                        help='Detection confidence threshold (0-1)')
    parser.add_argument('--output', type=str, default=None,
                        help='Path to save output video')
    parser.add_argument('--no-display', action='store_true',
                        help='Disable display (useful for headless systems)')

    args = parser.parse_args()
```

```
    print("=" * 60)
    print("Object Detection and Tracking - CodeAlpha AI Internship")
    print("=" * 60)
    print(f"\nModel: {args.model}")
    print(f"Confidence Threshold: {args.confidence}")
    print(f"Source: {'Webcam' if args.source == '0' else args.source}")

    # Initialize detector and tracker
    detector = ObjectDetectorTracker(
        model_name=args.model,
        confidence=args.confidence
    )

    # Determine source type
    source = int(args.source) if args.source.isdigit() else args.source

    # Check if source is an image
    if isinstance(source, str) and source.lower().endswith(('.jpg', '.jpeg',
'.png', '.bmp')):
        output = args.output or 'detected_' + os.path.basename(source)
        detector.process_single_image(source, output)
    else:
        # Process video
        detector.process_video(
            source=source,
            output_path=args.output,
            show=not args.no_display
        )

if __name__ == '__main__':
    main()
```

🚀 How to Run

1. Install dependencies: `pip install -r requirements.txt`
2. Run with webcam: `python main.py`
3. Run with video file: `python main.py --source video.mp4`
4. Save output: `python main.py --output tracked_output.mp4`
5. Use different model: `python main.py --model yolov8s.pt`

✨ **Features Implemented:**

- YOLOv8 for state-of-the-art object detection
- Custom SORT-like tracker with IoU matching
- Hungarian algorithm for optimal assignment
- Persistent track IDs across frames
- Real-time FPS display
- Support for webcam and video files
- Video output saving capability
- 80 COCO class detection

📚 Summary

This document provides complete, working solutions for all four AI tasks in the CodeAlpha internship program:

Task	Technology Stack	Key Features
Task 1: Language Translation	Flask, Google Translate API, gTTS	Auto-detect, 100+ languages, TTS, Copy
Task 2: FAQ Chatbot	Flask, NLTK, Scikit-learn	NLP preprocessing, TF-IDF, Cosine similarity
Task 3: Music Generation	TensorFlow, Keras, music21	3-layer LSTM, Temperature control, MIDI I/O

Task 4: Object Detection	OpenCV, YOLOv8, Custom SORT	Real-time tracking, Persistent IDs, 80 classes

📌 **Remember:** Complete at least 2-3 tasks to be eligible for the internship certificate. Upload your code to GitHub with the naming convention `CodeAlpha_ProjectName` and post a video explanation on LinkedIn.

Document prepared for CodeAlpha AI Internship Program

www.codealpha.tech